AULA 08

Manipulação de arquivos e tratamento de dados

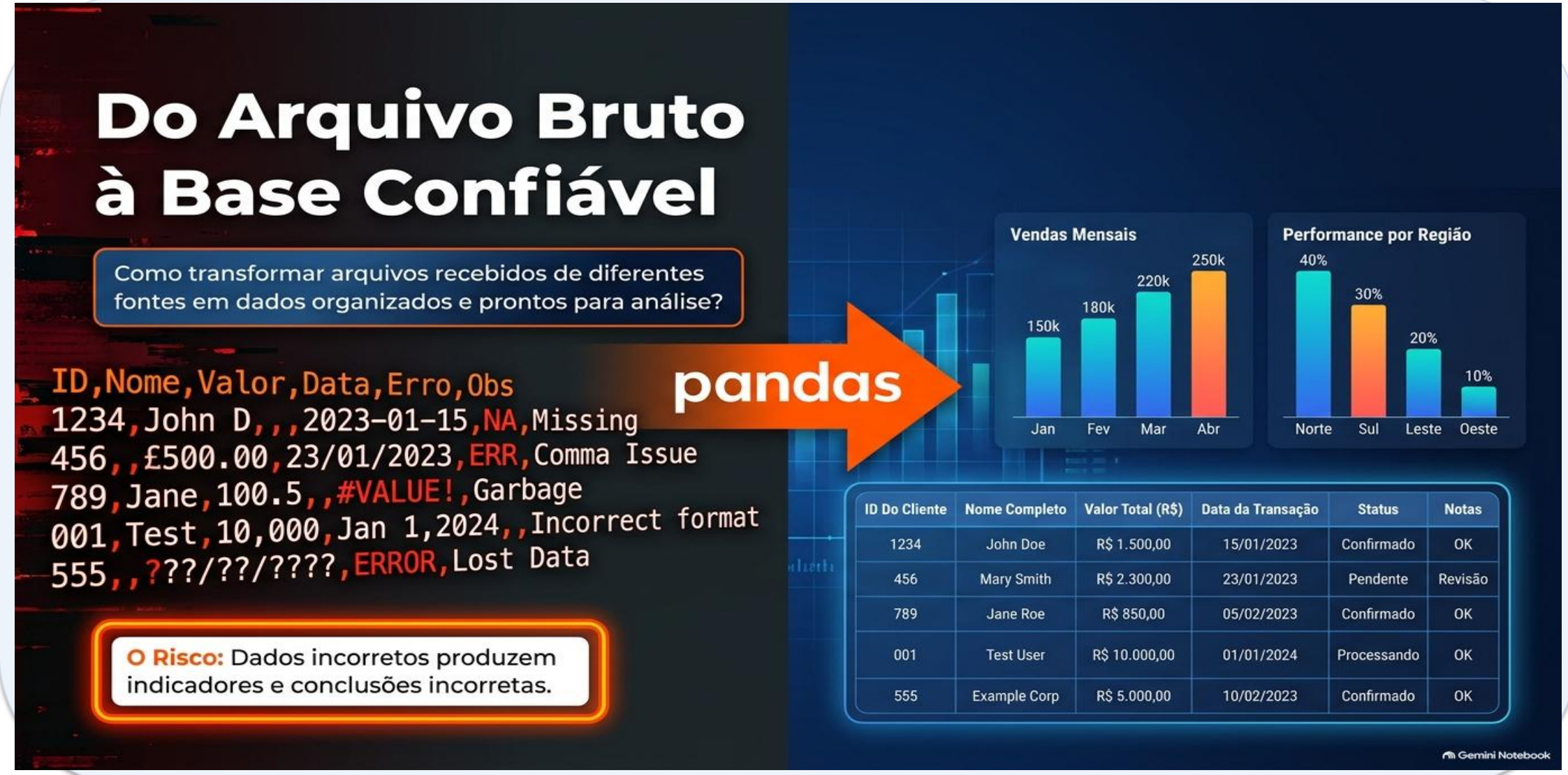
(CSV, Excel, pandas e limpeza de dados) – UC2

Do arquivo bruto à base confiável: importação, tratamento e preparação dos dados para análise.

PROF. ME. CARLOS DUARTE



Do arquivo bruto à base confiável



Do arquivo bruto à base confiável

Dados reais nem sempre chegam prontos para análise

O que acontece na prática?

Arquivos recebidos de diferentes fontes (sistemas, planilhas, exportações, formulários) podem vir com formatos diferentes, erros e informações inconsistentes.

Esses problemas dificultam a análise e podem levar a indicadores e conclusões incorretas, como mostra o exemplo ao lado.

Exemplo do slide

O arquivo de vendas contém valores faltantes (NA), erros (ERR), formatos incorretos e símbolos de moeda. Antes de calcular indicadores, é necessário tratar esses dados.

Principais causas

1. Dados digitados manualmente (sujeitos a erros).
2. Arquivos oriundos de fontes diferentes, com formatos variados.
3. Uso de separadores inconsistentes (vírgula, ponto e vírgula, tabulação).
4. Valores ausentes, duplicados ou com caracteres inesperados.
5. Campos numéricos armazenados como texto (ex.: “R$ 1.500,00”).

Problemas comuns em arquivos de dados

Valores ausentes
Ex.: NA, vazio, nulo. Podem distorcer médias e totais.

Erros de digitação
Ex.: ERR, caracteres inválidos, textos no lugar de números.

Formatos inconsistentes
Ex.: datas em diferentes padrões, uso de vírgula e ponto.

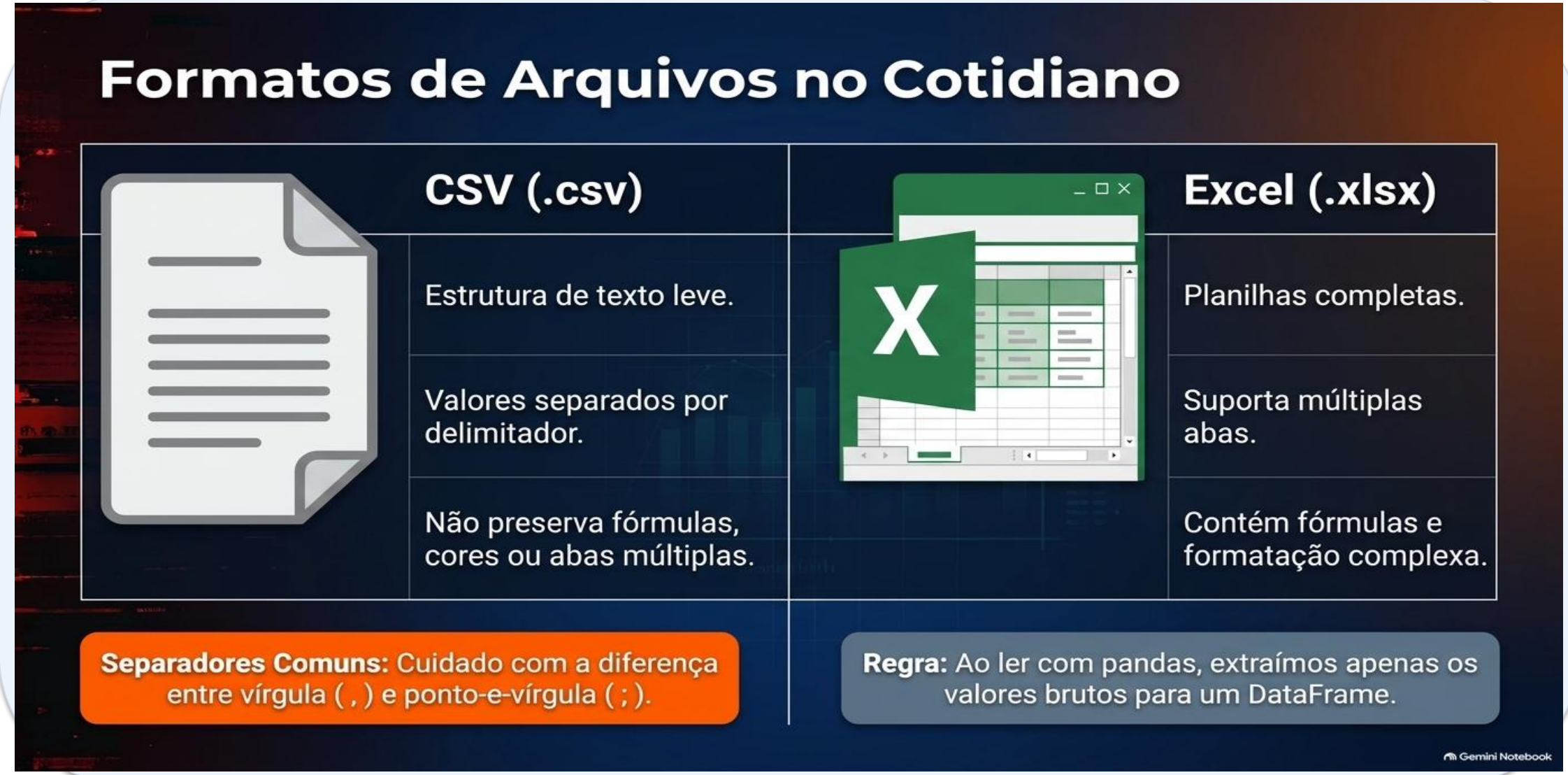
Símbolos e texto misturados
Ex.: “R$ 1.500,00” como texto, impossibilitando cálculos.

Registros duplicados
Podem inflar totais e afetar indicadores.

Por que tratar os dados?

- Garante cálculos corretos e indicadores confiáveis.
- Facilita a comparação entre períodos, regiões e categorias.
- Permite identificar padrões e tendências reais.
- Evita conclusões equivocadas e decisões baseadas em dados incorretos.

Formatos de arquivos no cotidiano



Trazendo os dados para a nuvem



Trazendo os dados para a nuvem

Como importar arquivos para o Google Colab de forma prática e segura

Passo a passo no Google Colab

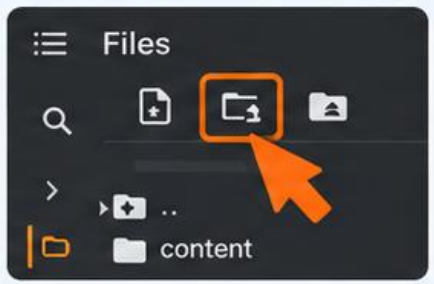
1 Abrir o painel lateral

No Colab, clique no ícone de pasta (🗀) no painel lateral esquerdo para acessar os arquivos.



2 Fazer upload do arquivo

Clique no ícone "Fazer upload para o armazenamento da sessão" e selecione o arquivo no seu computador.



3 Acessar o arquivo

O arquivo enviado fica na pasta temporária `/content/` e pode ser lido diretamente com o pandas.

```
import pandas as pd

# Ler o arquivo enviado
df = pd.read_csv('/content/vendas.csv')
```

O que é a pasta /content/?

É um espaço temporário de armazenamento do Google Colab. Os arquivos enviados ficam disponíveis apenas enquanto a sessão estiver ativa. Se a sessão for encerrada, os arquivos são perdidos e será necessário fazer o upload novamente.

Importante:
Salve seus arquivos localmente e mantenha a planilha original para refazer o upload quando necessário.

Dicas para um upload sem problemas

- Verifique se o arquivo está no formato correto (.csv ou .xlsx).
- Use nomes simples, sem espaços ou caracteres especiais.
- Confira o tamanho do arquivo (arquivos muito grandes podem demorar para enviar).
- Após o upload, utilize `df.head()` para confirmar que os dados foram carregados corretamente.

Abrindo arquivos com pandas



Abrindo arquivos com pandas

Como ler arquivos CSV e Excel no Google Colab

Arquivos CSV (.csv)

O formato CSV armazena os dados como texto, com os campos separados por um delimitador (geralmente a vírgula) e cada registro em uma linha.

```
import pandas as pd
# Ler arquivo CSV do Colab
df = pd.read_csv(
    '/content/vendas.csv',
    sep=',',            # separador de campos
    encoding='utf-8'    # codificação do arquivo
)
```

1. **sep=','**
   Indica qual é o separador de campos. O padrão é a vírgula.
2. **encoding='utf-8'**
   Define a codificação de caracteres, evitando problemas com acentuação (ç, á, ã, etc.).
3. **O resultado**
   O pandas cria um DataFrame com os dados do arquivo.

Arquivos Excel (.xlsx)

O formato Excel permite múltiplas planilhas (abas) em um mesmo arquivo. Precisamos informar qual aba será lida.

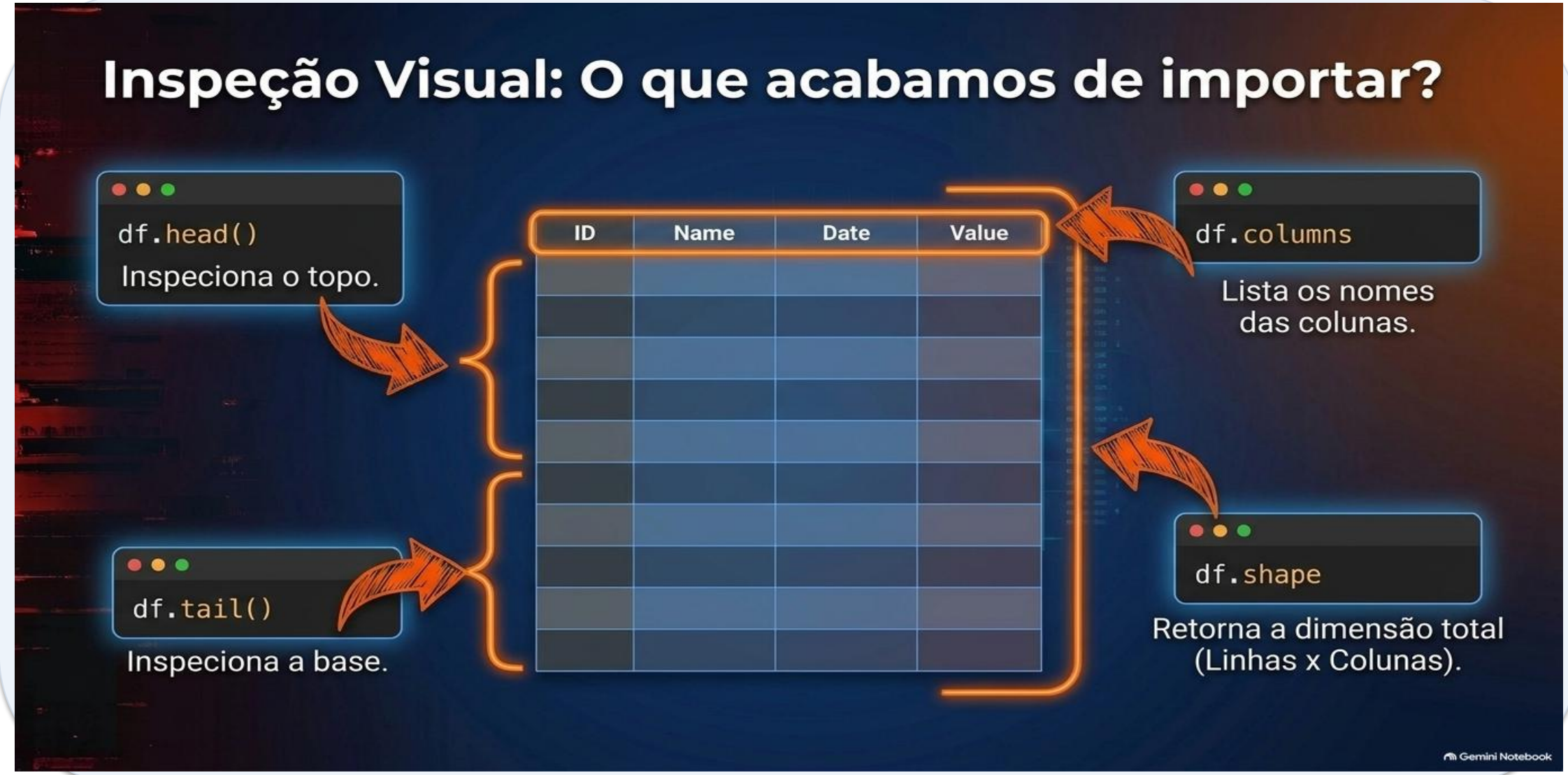
```
import pandas as pd
# Ler arquivo Excel do Colab
df = pd.read_excel(
    '/content/vendas.xlsx',
    sheet_name=0  # 0 = primeira aba (também pode ser o nome)
)
```

1. **sheet_name=0**
   Especifica a aba que será lida. Pode ser:
   - 0, 1, 2... (posição da aba)
   - o nome da aba (ex.: sheet_name='Vendas')
2. **Extensão .xlsx**
   Usa o motor de leitura do pandas para arquivos do Excel.
3. **O resultado**
   O pandas cria um DataFrame com os dados da aba selecionada.

Boas práticas

- Verifique se o arquivo está no local correto (/content/).
- Confirme o separador e a codificação (no CSV) quando houver erros de leitura.
- Após a leitura, use df.head() para conferir se os dados foram carregados corretamente.
- Em arquivos Excel com várias abas, verifique o nome das planilhas com pd.ExcelFile().sheet_names.

Inspeção visual dos dados



Inspeção visual dos dados

Funções básicas para conhecer a estrutura do DataFrame

Principais funções

1 df.head()

Mostra as primeiras linhas do DataFrame.

```
# Exibe as 5 primeiras linhas (padrão)
df.head()
```

2 df.tail()

Mostra as últimas linhas do DataFrame.

```
# Exibe as 5 últimas linhas (padrão)
df.tail()
```

3 df.columns

Lista o nome de todas as colunas.

```
# Mostra o nome das colunas
df.columns
```

4 df.shape

Retorna a dimensão do DataFrame.

```
# (linhas, colunas)
df.shape
```

Exemplo prático

Ao carregar o arquivo, usamos essas funções para verificar se os dados foram lidos corretamente.

```
import pandas as pd
df = pd.read_csv('/content/vendas.csv', sep=',,', encoding='utf-8')

# Inspeção visual
df.head()        # primeiras linhas
df.tail()        # últimas linhas
df.columns       # nomes das colunas
df.shape         # total de linhas e colunas
```

ID	Nome	Data	Valor	Região
1234	Ana	2023-01-15	1500.00	Norte
456	Bruno	2023-01-23	2300.00	Sul
789	Carla	2023-02-05	850.00	Leste
001	Daniel	2024-01-01	10000.00	Oeste
555	Ema	2023-02-10	5000.00	Norte

Por que fazer isso?

Essas funções ajudam a confirmar se o arquivo foi lido corretamente, identificar os nomes das colunas e entender o tamanho do conjunto de dados antes de começar a análise.

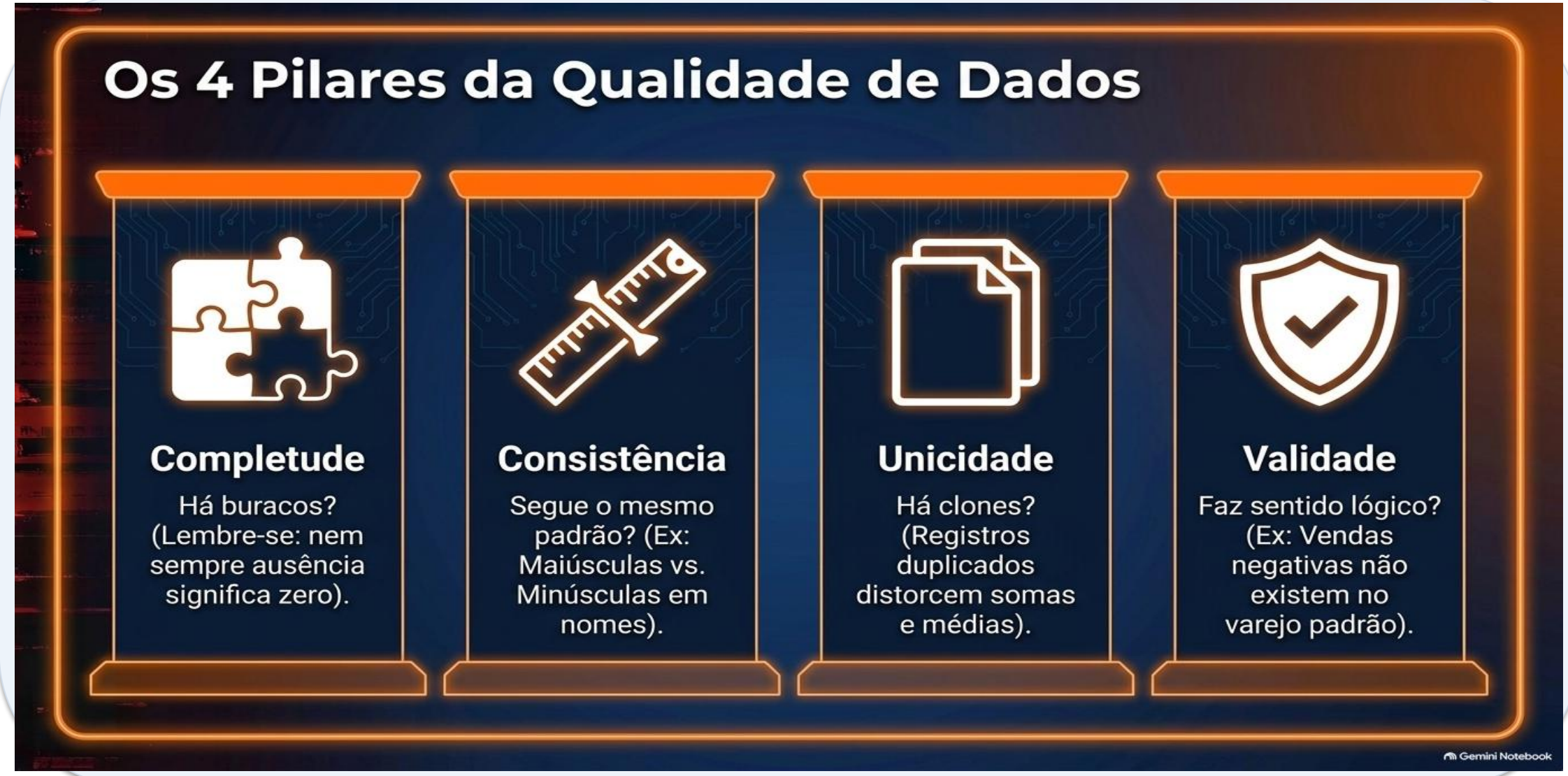
Dicas importantes

- Verifique se os nomes das colunas estão corretos.
- Observe se os dados fazem sentido (tipos e valores).
- Confirme se a quantidade de linhas e colunas é a esperada.
- Use essas verificações sempre que importar um novo arquivo.

Diagnóstico de integridade dos dados



Os quatro pilares da qualidade de dados



Os quatro pilares da qualidade de dados

Critérios essenciais para uma base confiável e pronta para análise

1. Completude

Todos os campos necessários estão preenchidos.

O que verificar?

- Existem valores ausentes (NaN)?
- Campos obrigatórios estão preenchidos?
- A quantidade de registros é a esperada?

Exemplo

Se a coluna “Valor” tiver muitos valores ausentes, a soma de vendas ficará incorreta.

2. Consistência

Os dados seguem o mesmo padrão e formato.

O que verificar?

- Mesma estrutura em todos os registros (datas, números).
- Uso consistente de maiúsculas e minúsculas.
- Padronização de categorias (ex.: “Norte” e não “norte”).

Exemplo

Datas no mesmo formato (2023-01-15) evitam erros de leitura e comparação.

3. Unicidade

Cada registro representa um item único.

O que verificar?

- Existem registros duplicados?
- O identificador (ID) é único para cada registro?
- Não há cópias do mesmo dado com pequenas diferenças (espaços, acentos).

Exemplo

Registros duplicados podem inflar os totais e distorcer as médias.

4. Validade

Os valores fazem sentido e estão dentro do esperado.

O que verificar?

- Valores negativos onde não deveriam existir.
- Números fora da faixa esperada.
- Textos inválidos ou com caracteres incorretos.
- Categorias desconhecidas (ex.: região inexistente).

Exemplo

Uma venda com valor negativo pode indicar erro de digitação.

Como aplicar na prática?

1. Importe o arquivo com **pandas**.
2. Use as funções de inspeção (**df.head()**, **df.info()**, **df.isnull()**, **df.duplicated()**, etc.).
3. Analise os resultados e identifique problemas em cada pilar.
4. Trate os dados antes de iniciar a análise (limpeza, padronização, remoção de duplicados).

Criando anomalias para testar

Como gerar dados propositalmente inconsistentes para praticar a limpeza

1. Criando um conjunto de dados com anomalias

Para praticar a identificação e o tratamento de problemas, podemos criar um pequeno conjunto de dados com erros intencionais.
No exemplo abaixo, incluímos um valor ausente (NaN) e registros **duplicados** para simular situações reais.

```
import pandas as pd
# Criando dados com anomalias intencionais
dados = {
    'Vendedor': ['Ana', 'Bruno', 'Ana', 'Carla', 'Bruno'],
    'Vendas': [1200, 850, None, 1600, 850], # valor ausente
    'Regiao': ['Norte', 'Sul', 'Norte', 'Sul', 'Sul']
}

df = pd.DataFrame(dados)
```

2. Resultado: dados com anomalias

Ao executar o código, obtemos um **DataFrame** com problemas que precisam ser identificados e corrigidos.

	Vendedor	Vendas	Regiao
0	Ana	1200.0	Norte
1	Bruno	850.0	Sul
2	Ana	NaN	Norte
3	Carla	1600.0	Sul
4	Bruno	850.0	Sul

Duplicados
O vendedor "Bruno" aparece duas vezes com os mesmos valores (linhas 1 e 4).

Valor ausente
A venda do terciero registro está como NaN.

Registro duplicado
O vendedor "Ana" aparece mais de uma vez (linhas 0 e 2).

Por que criar anomalias?

Trabalhar com dados "imperfeitos" ajuda a desenvolver habilidades para identificar e corrigir problemas comuns, como:

- Valores ausentes (NaN).
- Registros duplicados.
- Categorias inconsistentes (ex.: "Norte" e "norte").
- Valores fora do padrão (números em texto, datas inválidas).

Como usar esse exemplo na prática?

1. Execute o código para criar o DataFrame.
2. Use as funções de inspeção (df.head(), df.info(), df.isnull(), df.duplicated()).
3. Identifique as anomalias e proponha soluções de limpeza.
4. Compare o resultado antes e depois do tratamento dos dados.

Valores ausentes: identificar e decidir

Como detectar valores ausentes e escolher a melhor forma de tratar

Valores ausentes são representados no pandas como **NaN (Not a Number)**.
Eles podem ocorrer por falhas de registro, integração de fontes diferentes ou informações realmente desconhecidas.

1 Identificar valores ausentes

Use **df.isna()** para localizar os valores ausentes no DataFrame.

```
#  Verificar quantos valores ausentes há em cada coluna
df.isna().sum()
```

2 Remover registros com valores ausentes

Use **df.dropna()** para excluir as linhas que possuem NaN.

```
# Remove todas as linhas que têm pelo menos um NaN
df_limpo = df.dropna()
```

Atenção: Ao remover linhas, você perde dados. Use essa opção apenas quando a quantidade de registros excluídos for pequena.

3 Preencher valores ausentes

Use **df.fillna()** para substituir os NaN por um valor adequado.

```
# Substituir valores ausentes pela mediana da coluna 'Vendas'
mediana = df['Vendas'].median()
df['Vendas'] = df['Vendas'].fillna(mediana)
```

Quando usar?

- **Média:** quando a variável é numérica e não há muitos valores extremos.
- **Mediana:** quando há valores muito altos ou muito baixos (outliers).
- **Valor fixo:** quando faz sentido usar um padrão (ex.: 0, "Desconhecido").
- **Modo:** quando a variável é categórica (ex.: região, status).

Regra de Ouro:

Nunca preencha ou remova valores ausentes sem compreender o contexto dos dados e o impacto no seu negócio.

Identificação de registros duplicados

Como encontrar e remover registros duplicados para evitar distorções na análise

Registros duplicados são cópias do mesmo dado, que podem ocorrer por erros de cadastro, integração de fontes ou importações repetidas. Eles podem distorcer indicadores, somas e médias, por isso devem ser identificados e tratados.

1 Identificar registros duplicados

Use o método duplicated() para localizar as linhas que são cópias exatas de outras.

python

```python
# Verificando quais linhas são duplicadas
df_tratado.duplicated()
print(df_tratado.duplicated().sum())
```

Dados com duplicatas

ID	Vendedor	Vendas	Regiao
0	Ana	1200	Norte
1	**Bruno**	850	Sul
2	Ana	1200	Norte
3	Carla	1600	Sul
4	**Bruno**	850	Sul
5	Daniel	500	Leste

2 Remover registros duplicados

Use o método drop_duplicates() para manter apenas a primeira ocorrência.

python

```python
# Removendo as duplicatas (mantém a primeira)
df_tratado = df_tratado.drop_duplicates()
print(len(df_tratado))
```

O que o código faz?

Remove as linhas duplicadas e retorna o novo tamanho do DataFrame, mantendo a primeira ocorrência de cada registro.

Resultado após a remoção

O DataFrame fica apenas com registros únicos. O segundo "Bruno" foi removido.

Dados sem duplicatas

ID	Vendedor	Vendas	Regiao
0	Ana	1200	Norte
1	**Bruno**	850	Sul
3	Carla	1600	Sul
5	Daniel	500	Leste

Perceba a mudança

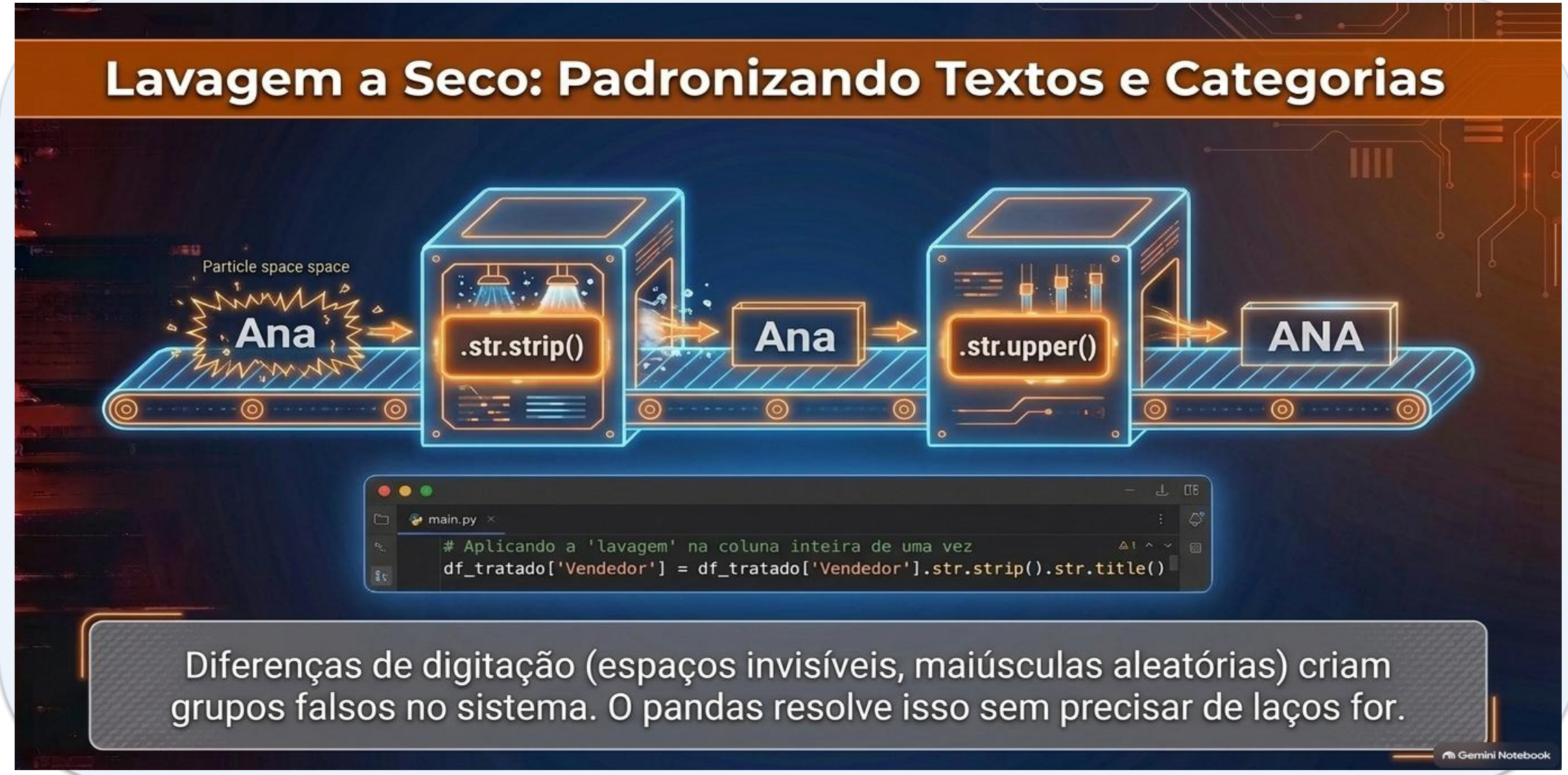
O vendedor "Bruno" agora aparece apenas uma vez, evitando que os valores sejam contabilizados duas vezes na análise.

Dicas importantes

- O método `drop_duplicates()` mantém apenas a primeira ocorrência por padrão.
- Você pode definir colunas específicas para considerar como duplicatas (ex.: `['Vendedor', 'Vendas']`).
- Registros duplicados podem não ser exatamente iguais (ex.: espaços, maiúsculas/minúsculas).
- Sempre investigue e padronize os dados antes de remover duplicatas.

Regra de Ouro: Registros diferentes na mesma data, na mesma loja ou com pequenas variações não devem ser removidos sem análise do contexto.

Padronização de textos e categorias



Padronização de textos e categorias

Como aplicar a “lavagem a seco” para padronizar e evitar categorias duplicadas

Dados de texto podem vir com diferenças de digitação, espaços extras e maiúsculas/minúsculas.
A “lavagem a seco” padroniza os textos para que valores iguais sejam reconhecidos como uma única categoria.

1 Identificar os problemas

Observe como o mesmo nome aparece de formas diferentes no DataFrame, gerando categorias duplicadas.

Dados antes da padronização

ID	Vendedor	Vendas	Regiao
0	Ana	1200	Norte
1	' ana '	850	Norte
2	ANA	950	Norte
3	Ana '	1100	Norte
4	Bruno	800	Sul

2 Aplicar a padronização

Usamos os métodos `.str.strip()` e `.str.upper()` para remover espaços e padronizar para maiúsculas.

```python
# 1. Remove espaços extras no início e no fim
df_tratado['Vendedor'] = df['Vendedor'].str.strip()

# 2. Converte todo o texto para maiúsculo
df_tratado['Vendedor'] = df_tratado['Vendedor'].str.upper()

# Resultado
print(df_tratado['Vendedor'].unique())
```

Sequência importante:

1. Remover espaços (`.str.strip()`)
2. Padronizar maiúsculas (`.str.upper()`)

3 Resultado após a padronização

Todos os registros “Ana” agora aparecem da mesma forma, sem duplicatas.

Dados após a padronização

ID	Vendedor	Vendas	Regiao
0	ANA	1200	Norte
1	ANA	850	Norte
2	ANA	950	Norte
3	ANA	1100	Norte
4	BRUNO	800	Sul

Agora são categorias únicas!

O nome “ANA” foi padronizado e aparece apenas uma vez no conjunto de dados.

Benefícios da padronização

- Evita categorias duplicadas.
- Melhora a qualidade das análises (contagens, somas, médias).
- Deixa os dados prontos para visualizações e modelos.

Regra de Ouro: Sempre padronize os campos de texto antes de analisar ou agrupar os dados. Isso evita distorções e garante resultados confiáveis.



OBRIGADO!

Foi um prazer ter você nesta aula.
Até a próxima!

Dúvidas e sugestões:
consultprofcebd@gmail.com

Prof. Me. Carlos Duarte